

International Institute of Information Technology, Hyderabad

Software Engineering

PHOENIX

Intelligent Personal Financial Analytics Platform

Team 23

Amritha Vinod — 2023101075

Ananya Halgatti — 2023101129

Janya Gupta — 2022102033

Kriti Gupta — 2023101123

Priyanshi Gupta — 2023101068

Code Repository: [GitHub Link](#)

April 2026

Domain: Financial Technology (FinTech)

Task 1 : Requirements and Subsystems

Functional and Non-functional Requirements

Requirements

Functional Requirements. Phoenix provides an intelligent personal finance interface that allows users to ingest transactions from bank APIs, CSV uploads, and manual entry. The platform normalizes every source into a common transaction schema, categorizes expenses, computes a Financial Health Score, detects anomalous spending, tracks budgets and goals, and presents financial insights through a React dashboard. It also supports authentication, role-based access control, immutable audit logging, and PDF/CSV report export for selected date ranges.

Non-Functional Requirements. The system targets sub-50 ms p95 latency on cache hits and under 600 ms on cache misses, sustaining over 300 RPS for up to 50,000 DAU equivalent. Security is enforced via TLS 1.3, AES-256 encryption at rest, JWT gateway authentication, and zero plain-text PII in logs. Reliability is achieved through cache fallback, health checks, and fault isolation, ensuring 100% read availability during partial outages with MTTR under 30 seconds. All financial operations are captured in an immutable 7-year audit log, and DPDP Act compliance is ensured through cryptographic erasure and per-user encryption keys. Maintainability and extensibility are supported via the Adapter pattern and OpenAPI contracts.

Architecturally Significant Requirements. The most architecturally significant requirements are transaction ingestion, categorization, Financial Health Score computation, anomaly detection, performance, and auditability. Transaction ingestion requires an Adapter pattern because bank APIs, CSV files, and manual input all use different formats. Categorization requires a Strategy pattern and feature flag because the rule-based engine and optional LLM classifier must be interchangeable. FHS computation drives cache-aside design because dashboard users need low-latency scores. Anomaly detection drives an Observer-style notification flow. Performance requirements motivate Redis and ClickHouse. Auditability constrains the database design through an append-only audit log.

Subsystem Overview

Phoenix is decomposed into seven subsystems, each with a single domain responsibility. This decomposition supports independent development, testing, and horizontal scaling.

1. API Gateway (nginx)

Role: Provides the single external entry point for the application.

Key Components: Reverse proxy, JWT validation middleware, TLS termination, rate limiter, WebSocket proxy.

Primary Functions: Route requests, enforce authentication, terminate TLS, apply rate limits, and forward WebSocket alert traffic.

2. Auth Service

Role: Manages identity, authentication, and role-based access control.

Key Components: FastAPI auth endpoints, bcrypt password hashing, JWT issuer,

PostgreSQL users table.

Primary Functions: Register users, authenticate users, issue/refresh JWTs, and validate role claims.

3. Transaction Ingestion Service

Role: Converts raw transaction inputs into a normalized financial transaction model.

Key Components: Adapter registry, CSV adapter, manual-entry adapter, bank API adapter, Pydantic validators, PostgreSQL writer.

Primary Functions: Validate input, transform source-specific formats, persist transactions, and trigger analytics.

4. Analytics Engine

Role: Computes financial metrics and aggregate views used by the dashboard and recommendation service.

Key Components: FHS calculator, category aggregator, trend analyzer, Redis cache writer, ClickHouse writer.

Primary Functions: Compute FHS, update aggregates, write analytical copies, invalidate caches, and publish analytics-complete events.

5. Anomaly Detection Service

Role: Detects unusual transactions and creates alerts.

Key Components: Z-score detector, rolling statistics store, alert publisher, Redis Welford-state cache.

Primary Functions: Subscribe to analytics completion, compute anomaly scores, suppress weak baselines, and generate alerts.

6. Recommendation Service

Role: Produces budget recommendations and goal projections.

Key Components: 50/30/20 rule engine, goal tracker, projection calculator, strategy selector.

Primary Functions: Recommend budgets, track goal progress, project savings/spending outcomes, and expose recommendation APIs.

7. Frontend Dashboard

Role: Provides the user-facing financial insight experience.

Key Components: React SPA, Recharts visualizations, React Query data layer, WebSocket alert client, auth module.

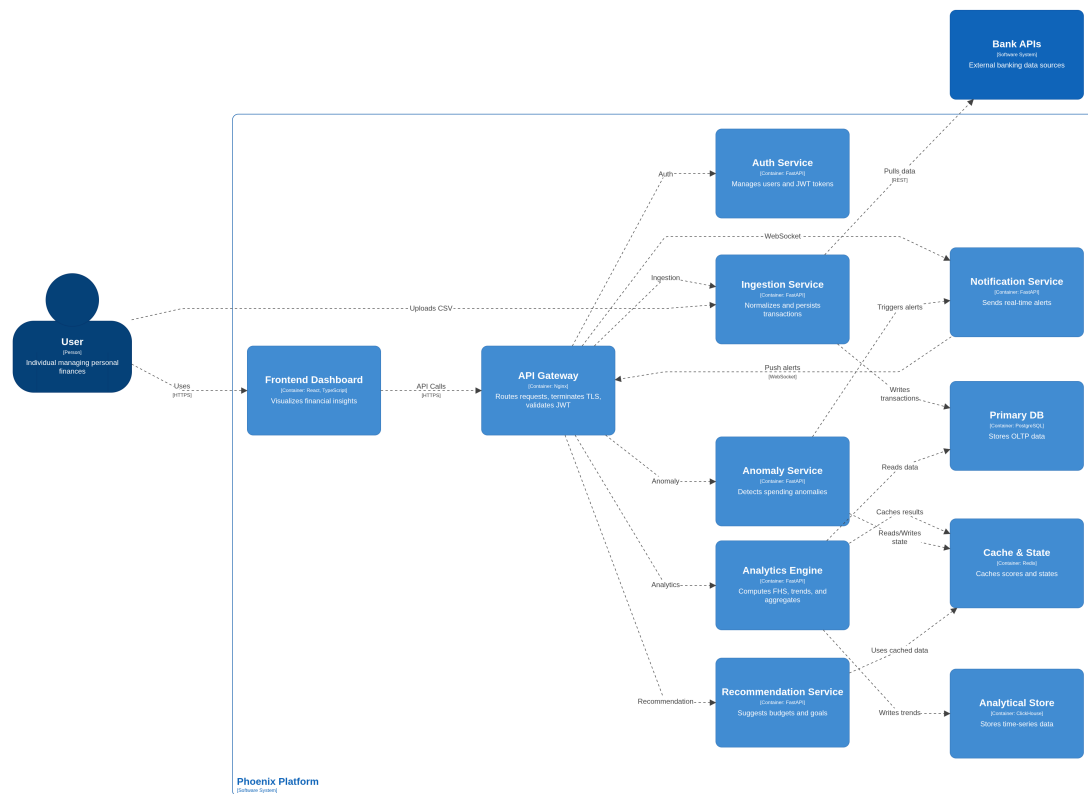
Primary Functions: Display FHS, spending trends, category breakdowns, transactions, alerts, budgets, and goals.

Subsystem Matrix

Subsystem	Responsibility	Key Components	Communication
1. API Gateway (nginx)	Single entry point. Routes requests, enforces JWT auth, handles TLS termination and rate limiting.	nginx reverse proxy; JWT validation middleware; rate limiter config	HTTP/REST to all backend services; WebSocket proxy to Notification Service
2. Auth Service	Issues and validates JWT tokens. Manages user registration, login, and RBAC.	FastAPI; bcrypt password hashing; JWT issuance; PostgreSQL users table	REST; called by Gateway on every request
3. Transaction Ingestion Service	Accepts raw transaction data, validates, normalizes via Adapter pattern, persists, and triggers downstream analytics.	Adapter registry; Pydantic schema validators; PostgreSQL writer; REST trigger to Analytics	REST inbound; REST outbound to Analytics Engine
4. Analytics Engine	On-demand computation of FHS, category distributions, and spending summaries. Writes results to PostgreSQL and invalidates Redis cache.	FHS Calculator; Category Aggregator; Redis cache writer; ClickHouse writer	Called synchronously by Ingestion; REST query by Recommendation Service
5. Anomaly Detection Service	Subscribes to analytics-complete notifications. Computes Z-score per transaction. Publishes alerts.	Z-score engine; rolling stats store (Redis); alert publisher	REST webhook from Analytics Engine; publishes to Notification Service
6. Recommendation Service	Generates per-user budget recommendations and goal projections on demand.	50/30/20 engine; Goal tracker; Strategy selector	REST; reads from PostgreSQL and Analytics Engine

Subsystem	Responsibility	Key Components	Communication
7. Frontend Dashboard	React SPA providing all user-facing visualizations, onboarding, and interaction.	Recharts; React Query; WebSocket client; Auth module	REST to API Gateway; WebSocket for live alerts

Major Requirement and Subsystem Diagrams



Container View: Phoenix Platform
Tuesday, April 21, 2020 at 1:02 AM India Standard Time

Figure 1: Subsystem / C4 Container Diagram showing API Gateway, frontend, backend services, PostgreSQL, Redis, ClickHouse, and external data sources.

Detailed Functional Requirements

Requirements are listed with their architectural significance: the requirements that drive service boundaries, data patterns, or technology choices are explicitly marked.

ID	Requirement	Description	Architecturally Significant?
FR-01	Multi-source Transaction Ingestion	Accept transactions via REST from bank API webhooks, CSV upload, and manual entry. Normalize via Adapter pattern to unified schema.	Yes: drives Adapter pattern; defines API Gateway routing
FR-02	Automated Expense Categorization	Classify each transaction using a rule-based engine (merchant name, MCC code, keyword matching). An LLM fallback layer is available via feature flag for ambiguous transactions.	Yes: Analytics microservice; feature-flag architecture for LLM toggle
FR-03	Financial Health Score (FHS)	Compute a composite score (0–100) from savings rate, debt-to-income ratio, spending volatility, and emergency fund ratio. Recomputed synchronously after each ingestion batch.	Yes: drives caching strategy; core analytics computation
FR-04	Budgeting Assistant	Generate monthly per-category budget recommendations using the 50/30/20 rule.	Moderate: Recommendation Service; REST query pattern
FR-05	Fraud / Anomaly Detection	Detect anomalous transactions using per-user per-category Z-score with rolling 30-day baseline. Alert threshold at $ Z > 2.5$.	Yes: Observer pattern; Notification Service integration
FR-06	Financial Insights Dashboard	React SPA showing FHS gauge, spending trend charts, category pie/bar, transaction list, and alert history.	Moderate: drives API contract; WebSocket for real-time alerts
FR-07	Goal Tracking	Users set savings/spending goals; system tracks progress and shows projections.	Low-Moderate: extends Recommendation Service
FR-08	Audit Log	Immutable append-only log of all data-processing operations: ingestion, categorization, FHS computation, and alerts.	Yes: append-only data design; PostgreSQL trigger enforcement
FR-09	User Authentication & Profile	JWT-based registration, login, and profile management. RBAC: User, Advisor, Admin roles.	Yes: cross-cutting security concern; API Gateway middleware
FR-10	Report Export	Export PDF/CSV spending summaries for a selected date range.	Low: peripheral; no architectural impact

Detailed Non-Functional Requirements

ID	Category	Requirement & Metric	Architectural Tactic
NFR-01	Performance	Dashboard API p95 latency < 50 ms (cache hit) and < 600 ms (cache miss) under production-like load.	Redis cache-aside; pre-aggregated metrics; ClickHouse-backed aggregation
NFR-02	Scalability	Sustain > 300 RPS throughput and scale horizontally to support high-concurrency workloads (50,000 DAU equivalent).	Microservices horizontal scaling; stateless services; ClickHouse for analytical reads
NFR-03	Security	TLS 1.3 in transit; AES-256 at rest; zero plain-text PII in logs.	JWT auth at gateway; column-level encryption; PII scrubber in log pipeline
NFR-04	Reliability	100% availability for read APIs during partial service outages; MTTR < 30 seconds for single-container failure.	Cache fallback (stale reads); Docker health checks; rapid container restart; fault isolation
NFR-05	Auditability	100% of financial operations in immutable audit trail, 7-year retention.	Append-only <code>audit_log</code> ; PostgreSQL INSERT-only trigger
NFR-06	Maintainability	New transaction source integrated in < 2 developer-days.	Adapter pattern; OpenAPI specs; consistent service contracts
NFR-07	Usability	Onboarding < 3 minutes; dashboard usable without documentation.	Progressive disclosure UI; sensible defaults; contextual tooltips
NFR-08	Privacy / Compliance	DPDP Act compliant; user data anonymizable within 72 hours on request.	Per-user encryption keys; cryptographic erasure; soft-delete

Task 2 : Architecture Framework

Stakeholder Identification

Stakeholder	Role	Primary Concerns	Viewpoint
End Users (Individuals)	Primary platform consumers	Accuracy of insights; data privacy; performance; mobile usability	Functional View; Security View
Financial Advisors	Secondary users reviewing client dashboards	Report accuracy; export quality; auditability of data	Data View; Functional View

Stakeholder	Role	Primary Concerns	Viewpoint
Banking / API Partners	External transaction data sources	API compatibility; schema standards; rate limits; SLA	Connector View; Deployment View
Development Team (Team 23)	Builders and maintainers	Modularity; testability; clear service contracts; CI/CD pipeline	Component & Connector View; Code View
Course Instructors	Academic evaluators	Pattern application; architectural rigor; documentation completeness; demonstrated prototype	All Views
Compliance / Privacy Officer	Regulatory adherence	Encryption; PII handling; audit log completeness; DPDP Act compliance	Security View; Data View
Operations (DevOps)	Deploy and monitor the system	Deployability via Docker Compose; health checks; observability; rollback	Deployment View; Runtime View

Viewpoint	Concern Addressed	View / Artifact
Functional	What does the system do? What are the subsystems?	Subsystem Overview; FR/NFR tables
Component & Connector	How do runtime services communicate?	C4 Container diagram; sequence diagrams; API contract tables
Data	How is data structured, stored, and moved?	ER schema; cache key table; data model table
Deployment	Where does the system run? How is it scaled?	Docker Compose manifest and deployment diagram
Security	How is the system secured and audited?	Security tactics; JWT flow; encryption and audit-log controls
Development	How is code organized? What are module boundaries?	Repository structure; OpenAPI spec references

Major Stakeholder Views

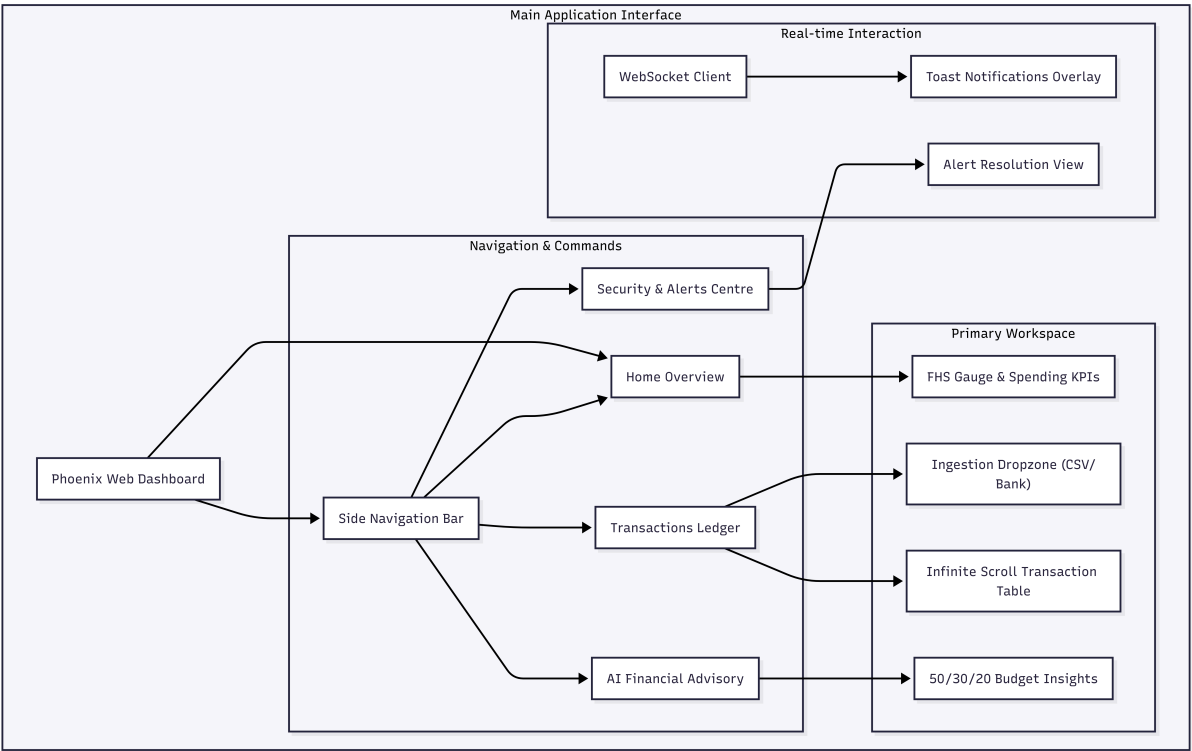


Figure 2: User View showing the user interacting with the Phoenix dashboard, transaction ingestion, budget recommendations, alerts, and report export.

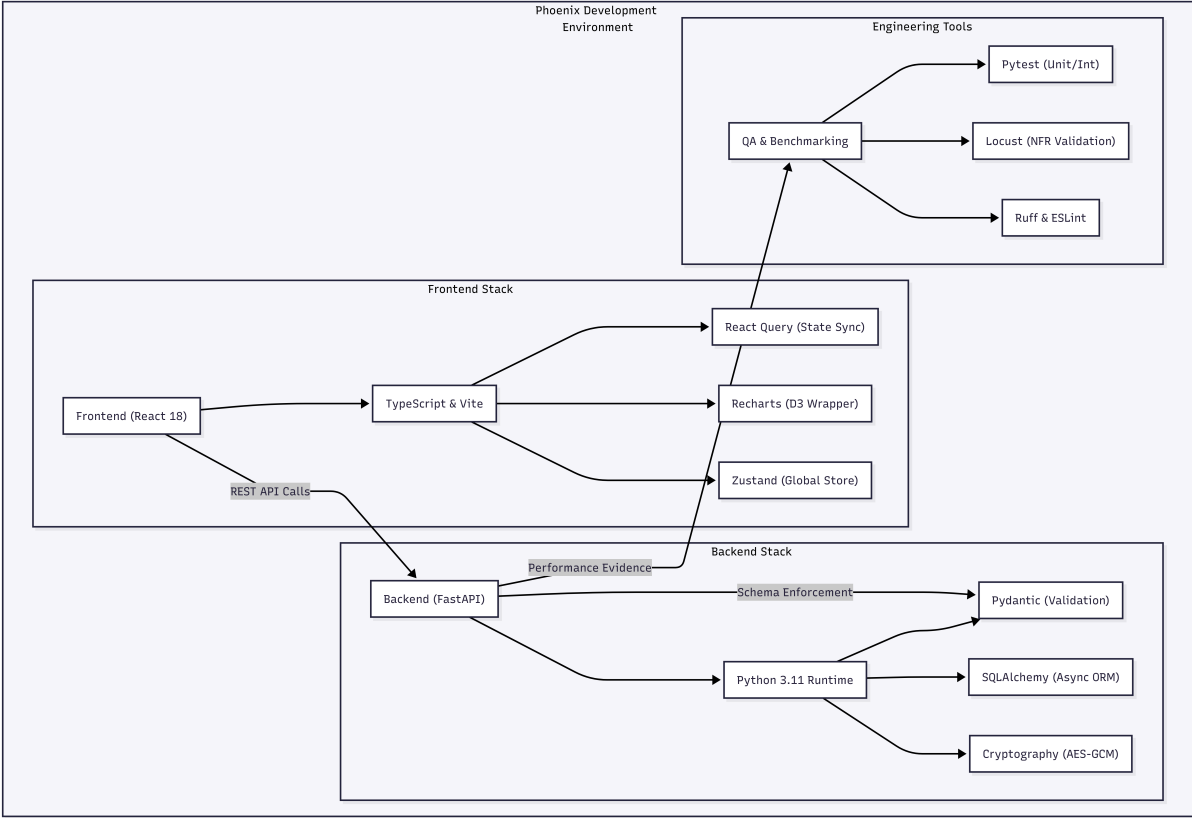


Figure 3: Developer View showing repository modules, service ownership, API contracts, adapter interfaces, analytics processors, and shared schemas.

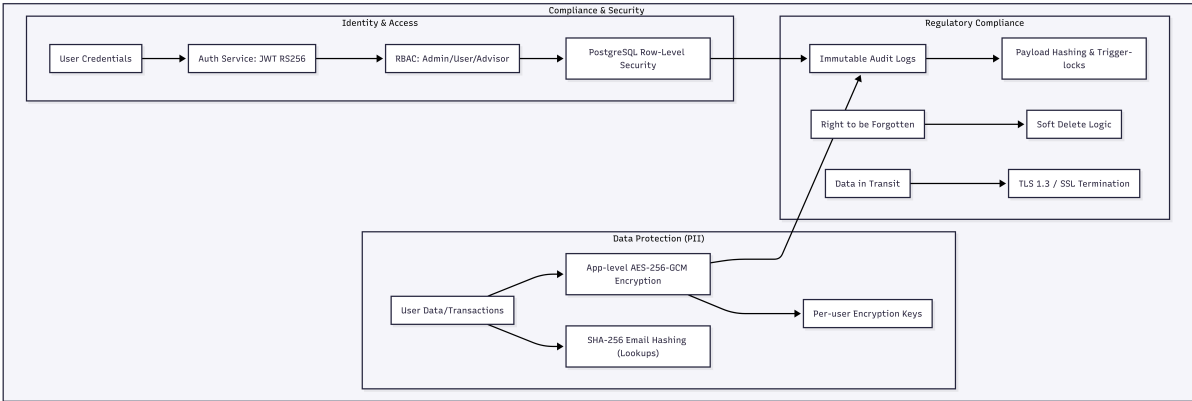


Figure 4: Compliance and Security View showing PII encryption, JWT validation, RBAC, audit logging, DPDP deletion flow, and log scrubbing.

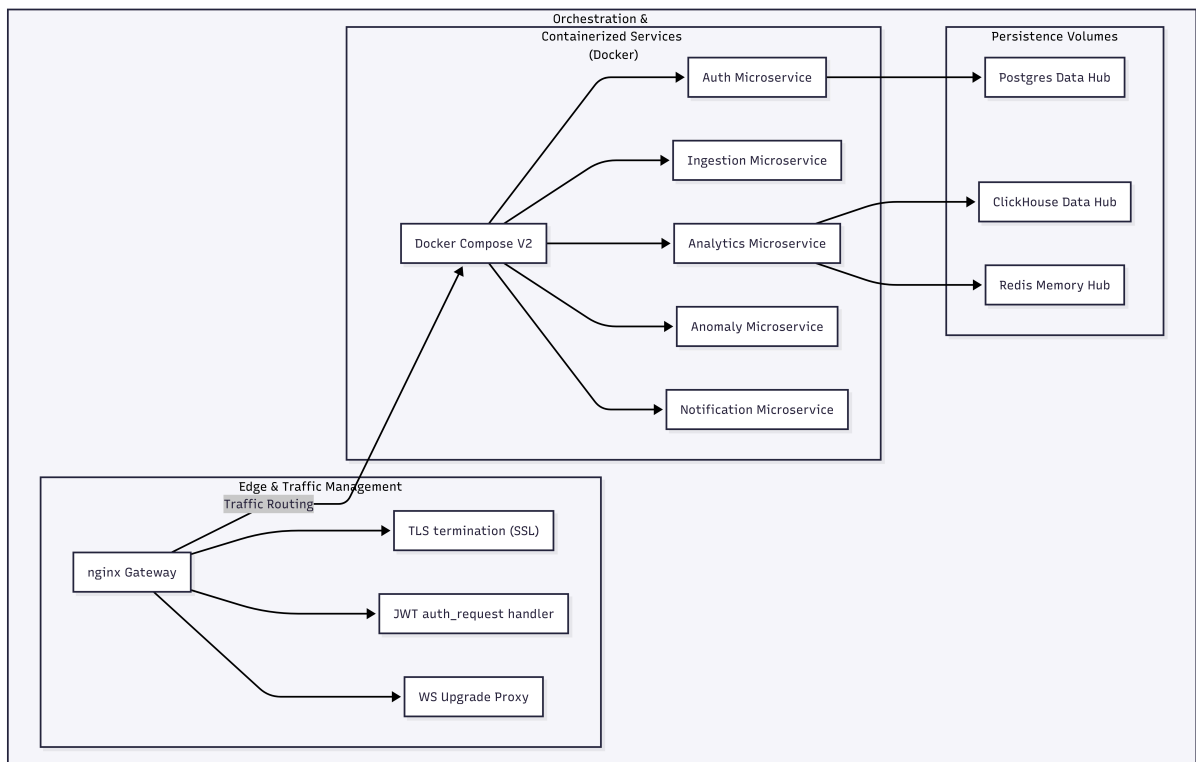


Figure 5: Deployer View showing Docker Compose services, nginx, internal network communication, PostgreSQL, Redis, ClickHouse, and health checks.

Major Design Decisions

All major architectural decisions are documented using the Nygard ADR template. Each ADR records the decided approach and its consequences.

ADR-001: Service Communication - REST-First with Optional Event Extension

Status: Accepted

Date: April 2026

Authors: Team 23

Context The system requires an inter-service communication strategy that is simple enough for local development while still supporting future scalability. The prototype must be easy to deploy, debug, and test, but the architecture should not prevent a shift to asynchronous event-driven processing if transaction volume increases. We evaluated pure synchronous REST, Kafka-first event-driven communication, and a hybrid approach.

Decision

Chosen Approach: Adopt a REST-first communication model for the prototype. All inter-service calls use synchronous HTTP/REST via the API Gateway. Certain interactions, such as ingestion-to-analytics and analytics-to-anomaly notification flows, are designed in a way that they can be extended to asynchronous messaging (e.g., Kafka) in the future without major interface changes.

Rationale

- REST-based communication minimizes infrastructure requirements and keeps the system easy to run in a local development environment.
- Synchronous request chains simplify debugging, logging, and traceability across services.
- Designing clear boundaries between services allows future migration to an event-driven architecture without redesigning APIs.
- A hybrid-ready design avoids premature optimization while keeping the system extensible.

Consequences

- Positive: simple deployment and lower operational overhead.
- Positive: easier debugging and end-to-end request tracing.
- Positive: clear service contracts through REST APIs.
- Negative: synchronous communication introduces temporal coupling under high load.
- Future consideration: asynchronous messaging (e.g., Kafka) can be introduced to decouple services and improve scalability if required.

Alternatives Considered

- **Pure REST:** Simplest approach, but limited scalability for high-volume systems.
- **Kafka-first:** Highly scalable, but introduces significant infrastructure and debugging complexity for the prototype.
- **Message queue only:** Useful for decoupling, but not ideal for synchronous dashboard interactions.

ADR-002: Database Architecture - PostgreSQL + Redis + ClickHouse

Status: Accepted

Date: April 2026

Authors: Team 23

Context Financial data involves two distinct access patterns: transactional operations requiring strong consistency (OLTP), and analytical queries over large datasets (OLAP). Additionally, the system must meet strict performance requirements for dashboard responsiveness.

Decision

Chosen Approach: Adopt a polyglot persistence strategy using PostgreSQL as the primary transactional database, Redis as a caching layer, and ClickHouse for analytical workloads. PostgreSQL stores user data, transactions, and audit logs. Redis stores frequently accessed aggregates such as Financial Health Scores and category summaries. ClickHouse stores time-series analytical data for trend analysis and reporting.

Rationale

- PostgreSQL provides ACID guarantees and strong relational integrity, essential for financial data.
- Redis enables low-latency access to frequently queried dashboard data.
- ClickHouse efficiently supports large-scale aggregation queries and time-series analysis.
- Separating OLTP and OLAP workloads prevents performance degradation.

Consequences

- Positive: reliable and consistent transactional storage.
- Positive: improved performance through caching and workload separation.
- Positive: scalable analytical queries without impacting core operations.
- Negative: increased operational complexity due to multiple data stores.

Alternatives Considered

- **PostgreSQL only:** Simpler architecture, but poor scalability for analytical queries.
- **MongoDB only:** Flexible schema, but weaker guarantees for financial integrity.
- **PostgreSQL + Redis only:** Simpler, but insufficient for large-scale analytical workloads.

ADR-003: Transaction Categorization — Rule-Based Engine with Optional LLM (Configuration-Controlled)

Status: Accepted

Date: April 2026

Authors: Team 23

Context The system must automatically categorize transactions efficiently while remaining accurate and maintainable. A purely rule-based system is fast but may struggle with ambiguous descriptions, while LLM-based approaches improve accuracy but introduce latency and external dependencies. The LLM-based enhancement is implemented but controlled via environment configuration and is not exposed as a runtime user-facing toggle.

Decision

Chosen Approach: Use a rule-based categorization engine as the primary classifier, based on MCC codes, merchant mappings, and keyword matching. A secondary LLM-based classifier is conditionally invoked via a backend feature flag (environment configuration) for low-confidence or ambiguous transactions.

Rationale

- Rule-based classification is fast, deterministic, and easy to debug.
- It avoids dependency on external services for core functionality.
- An LLM-based layer could improve classification accuracy for edge cases if integrated in the future.
- Feature flags allow flexible deployment without architectural changes.

Consequences

- Positive: system functions fully without LLM dependency.
- Potential: improved accuracy for ambiguous inputs if an LLM layer is added in the future.
- Trade-off: introducing an LLM would add latency and external API dependency.
- Limitation: rule-based system may misclassify complex transactions.
- Limitation: LLM path has not been fully tested due to external API constraints, though integration is implemented.

Alternatives Considered

- **LLM-only:** Higher accuracy, but slower and cost-intensive.
- **Traditional ML models:** Require training pipelines and maintenance.
- **Manual categorization:** Poor scalability and user experience.

ADR-004: Anomaly Detection — Statistical Z-Score with Rolling 30-Day Baseline

Status: Accepted

Date: April 2026

Authors: Team 23

Context The system must detect anomalous financial transactions in real time to support fraud detection and user alerts. The solution must operate effectively from early usage stages, without requiring large amounts of historical data or a training phase. It should also be computationally efficient to support real-time processing and produce interpretable results so that users can understand why a transaction was flagged.

More complex machine learning approaches provide higher accuracy but require training data, model management, and additional infrastructure. A simpler statistical approach is more suitable for the prototype stage.

Decision

Chosen Approach: Implement anomaly detection using a statistical Z-score method with a rolling 30-day baseline for each user and transaction category. Transactions are flagged as anomalies when their deviation from the baseline exceeds a predefined threshold.

Rationale

- Z-score-based detection requires no prior training and works with limited historical data.
- The method is computationally efficient and suitable for real-time processing.
- It provides interpretable results, allowing alerts to be explained in terms of deviation from normal behavior.
- A rolling baseline ensures that the system adapts to changing user spending patterns over time.
- The approach keeps the system simple while leaving room for future extensions.

Consequences

- Positive: no cold-start problem since no training phase is required.
- Positive: low computational overhead enables real-time detection.
- Positive: anomaly explanations are intuitive and user-friendly.
- Negative: limited to detecting anomalies based on transaction amounts.
- Limitation: does not capture multi-dimensional fraud patterns such as location, merchant behavior, or temporal anomalies.

Alternatives Considered

- **Isolation Forest:** More powerful for multi-dimensional anomaly detection but requires model training and lifecycle management.
- **Rule-based thresholds:** Simple but not adaptive to individual user behavior.
- **No anomaly detection:** Simplifies the system but removes an important user-facing feature.

Architecture Framework Diagrams

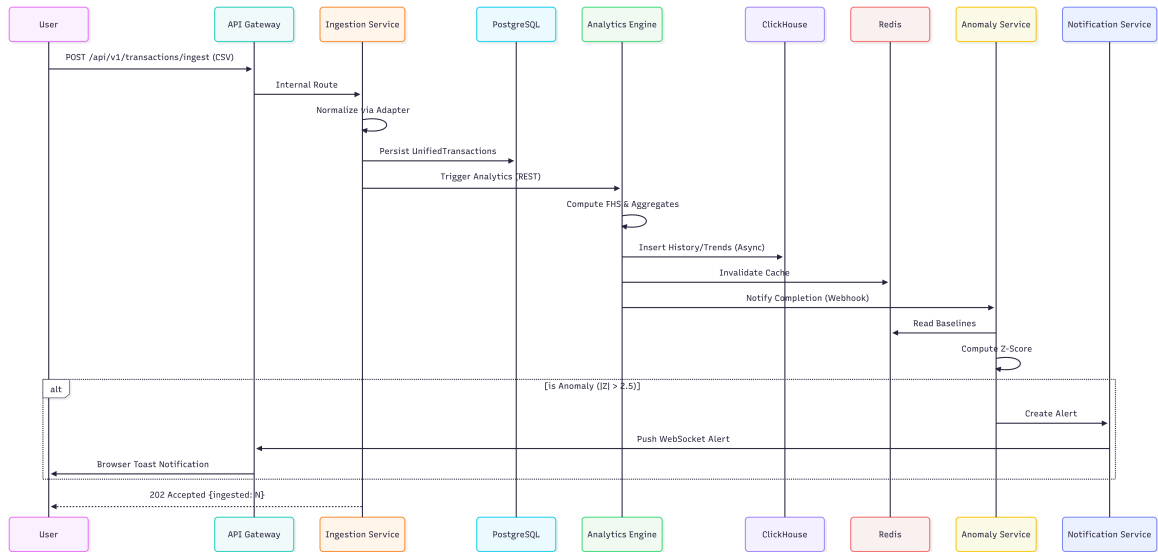


Figure 6: Sequence Diagram: Transaction Ingestion to Alert showing CSV/manual upload, gateway routing, ingestion adapter transformation, PostgreSQL write, analytics computation, cache invalidation, anomaly webhook, and WebSocket alert push.

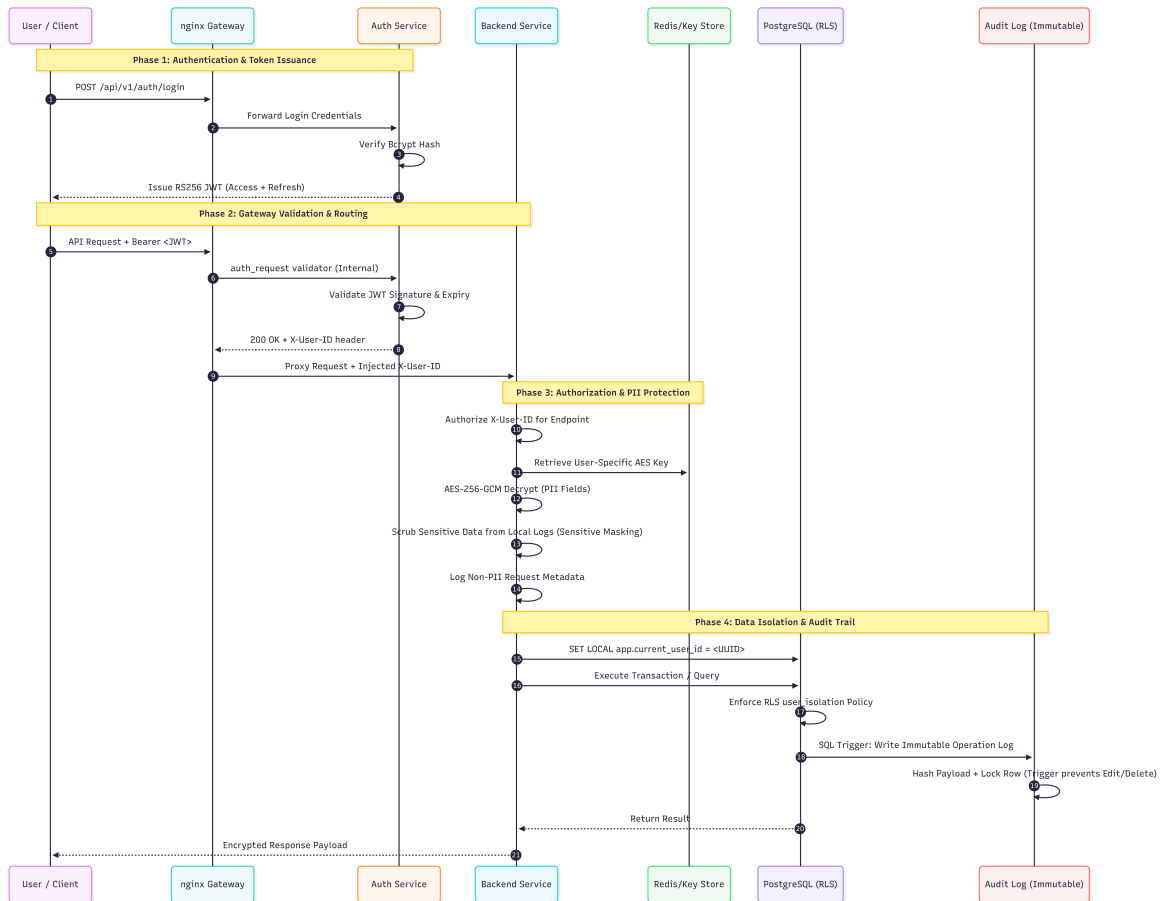


Figure 7: Security View: JWT and PII Protection Flow showing login/token issuance, gateway validation, service authorization checks, application-level encryption, log scrubbing, and audit-log write path.

Task 3: Architectural Tactics and Patterns

Architectural Tactics

Five tactics are employed, each directly addressing one or more NFRs. The implementation approach for each tactic is described in concrete Phoenix terms.

Tactic 1: Cache-Aside (Performance — NFR-01)

NFR Addressed

NFR-01: Dashboard API p95 latency < 50 ms (cache hit) and < 600 ms (cache miss) under production-like load.

All high-read computed artifacts, including the Financial Health Score, category distribution aggregates, and dashboard overview responses, are cached in Redis using the cache-aside pattern. The dashboard endpoint checks Redis first; on a cache miss, it queries PostgreSQL/ClickHouse, computes the result, writes to Redis with a TTL, and returns the response. Cache invalidation is event-driven: when the Analytics Engine finishes a computation cycle, it explicitly evicts the relevant user keys from Redis.

Cache Key	TTL	Invalidation Trigger	Backing Store
fhs:{user_id}	60s	Analytics Engine computation complete	PostgreSQL financial_health_scores
cat_dist:{user_id} :{YYYY-MM}	300s	New transaction in that month	ClickHouse category aggregates
dashboard:{user_id} :overview	30s	Any new transaction for user	Composite: FHS + category + alerts
anomaly_stats:{user_id} :{category}	No TTL; LRU eviction	New transaction updates stats inline	Redis Welford state
llm_category: {description_hash}	86400s (24 h)	Manual admin flush only	OpenAI API response

Tactic 2: Layered Security (Security — NFR-03)

NFR Addressed

NFR-03: TLS 1.3 in transit; AES-256 at rest; zero plain-text PII in logs.

Security is implemented in concentric layers:

- **Transport Layer:** nginx terminates TLS 1.3 for all external traffic. Internal Docker network traffic is HTTP within the trusted Docker bridge network.
- **Authentication Layer:** The API Gateway validates the JWT on every request before routing to a backend service. Token signing uses RS256.
- **Authorization Layer:** Each service checks the JWT role claim. PostgreSQL row-level security ensures users can only access their own rows.
- **Data Encryption:** Sensitive columns such as `account_number` and `raw_description` are encrypted at application level using AES-256-GCM before insertion.
- **PII Log Scrubbing:** Logging middleware replaces email, account number, and IBAN patterns with SHA-256 hashed tokens.

Tactic 3: Circuit Breaker (Reliability — NFR-04)

NFR Addressed

NFR-04: 100% availability for read APIs during partial failures (via stale cache); MTTR < 30 seconds for single-container failures.

Python’s `tenacity` library implements circuit breaker behavior around all synchronous inter-service REST calls. The circuit breaker wraps calls from Ingestion to Analytics and from the API Gateway to backend services. Configuration: 5 failures in 30 seconds opens the circuit; 60 seconds before half-open probe; probe success closes the circuit.

When the Analytics Engine circuit is open, the dashboard returns the last cached FHS with a `data_freshness: stale` flag. The frontend renders a subtle “last updated *X* minutes ago” indicator instead of failing the dashboard.

Tactic 4: Health Check & Self-Healing (Reliability — NFR-04)

Every service exposes `GET /health/live` for liveness and `GET /health/ready` for readiness. Docker Compose `healthcheck` directives poll these endpoints and restart containers that fail liveness checks. nginx upstream health checks route traffic away from unready service instances.

Tactic 5: Append-Only Audit Log (Auditability — NFR-05)

NFR Addressed

NFR-05: 100% of financial operations in immutable audit trail, 7-year retention.

The `audit_log` table in PostgreSQL is enforced as `INSERT`-only via a database trigger that raises an exception on any `UPDATE` or `DELETE`. Every service writes an audit record when it processes a financial event. Each record includes `operation`, `entity_type`, `entity_id`, `actor`, `timestamp`, and a SHA-256 hash of the payload to detect tampering. The audit log is only exposed through an admin-only paginated endpoint.

Implementation Tactics

Five GoF patterns are applied with concrete implementation details. The Adapter and Observer patterns are the most architecturally significant.

Pattern 1: Adapter — Transaction Source Integration

Bank APIs, CSV exports, and manual entry all produce different data formats. The Adapter pattern normalizes them to a single `UnifiedTransaction` schema.

Component	Type	File Location	Responsibility
<code>ICICIBankAdapter</code>	Concrete Adapter	<code>services/ingestion/adapters/icici_adapter.py</code>	Transforms ICICI Bank API JSON to <code>UnifiedTransaction</code>
<code>CSVUploadAdapter</code>	Concrete Adapter	<code>services/ingestion/adapters/csv_adapter.py</code>	Parses user-uploaded CSV with configurable column mapping
<code>ManualEntryAdapter</code>	Concrete Adapter	<code>services/ingestion/adapters/manual_adapter.py</code>	Validates and passes through manually entered transactions
<code>AdapterRegistry</code>	Registry	<code>services/ingestion/adapters/registry.py</code>	Maps <code>source_type</code> to <code>ITransactionAdapter</code> instance
<code>ITransactionAdapterInterface</code>		<code>services/ingestion/adapters/base.py</code>	Abstract methods: <code>validate(raw)</code> and <code>transform(raw)</code>

Component	Type	File Location	Responsibility
IngestionService	Client	services/ingestion/ service.py	Calls registry.get_adapter(...) .transform(raw)

Architectural guarantee: adding a new bank integration requires implementing `ITransactionAdapter` and registering it in `AdapterRegistry`; no existing service code changes are required. This directly satisfies NFR-06.

Pattern 2: Observer — Analytics-to-Anomaly Notification

When the Analytics Engine finishes computing the FHS update for a batch of transactions, it notifies the Anomaly Detection Service without tight coupling. The Observer pattern is implemented as a REST webhook through a `NotificationPublisher` abstraction.

Observer Role	Component	Event	Action
Subject	Analytics Engine	Emits <code>AnalyticsCompleteEvent</code> after FHS computation	Calls <code>NotificationPublisher.publish(event)</code>
Observer 1	Anomaly Detection Service	POST /internal/events/ analytics-complete	Computes Z-score for new transactions and creates alerts
Observer 2	Cache Invalidation Worker	POST /internal/events/ analytics-complete	Evicts Redis keys for the affected user
Publisher Abstraction	<code>RestWebhookPublisher</code>	Implements <code>INotificationPublisher</code>	HTTP POST to registered observer URLs from config
Future Publisher	<code>KafkaPublisher</code> (stub)	Implements <code>INotificationPublisher</code>	Publishes to <code>analytics.events</code> ; swap via <code>NOTIFICATION_BACKEND=kafka</code>

Pattern 3: Strategy — Transaction Categorization

The categorization strategy is selected at runtime based on feature flag and confidence score. Both strategies implement `ICategorizer(description, mcc_code, merchant) -> CategoryResult`.

- **RuleBasedCategorizer:** Primary strategy. Always active. Confidence depends on match type: MCC exact = 0.95, merchant exact = 0.90, keyword match = 0.70, no match = 0.0 to `Other`.
- **LLMCategorizer:** Secondary strategy. Active only when `ENABLE_LLM_CATEGORIZATION=true` and rule confidence is below 0.7. Sends description to `gpt-4o-mini` and caches the result for 24 hours.

Pattern 4: Factory Method — Analytics Service Instantiation

The Analytics Engine uses a factory to instantiate the appropriate analytics processor based on the event type received. `AnalyticsServiceFactory.create(event_type)` returns the correct processor (`FHSProcessor`, `CategoryAggregator`, or `TrendAnalyzer`) without the engine orchestrator knowing the concrete types.

Pattern 5: Facade — API Gateway Dashboard Endpoint

GET /api/v1/dashboard/overview is a Facade endpoint implemented in the API Gateway. It checks Redis first; on a miss it calls the Analytics Engine for FHS, Recommendation Service for the top budget alert, and PostgreSQL for the five most recent transactions. The frontend receives a single JSON response and does not need to know about the underlying services.

Implementation Tactic Diagrams

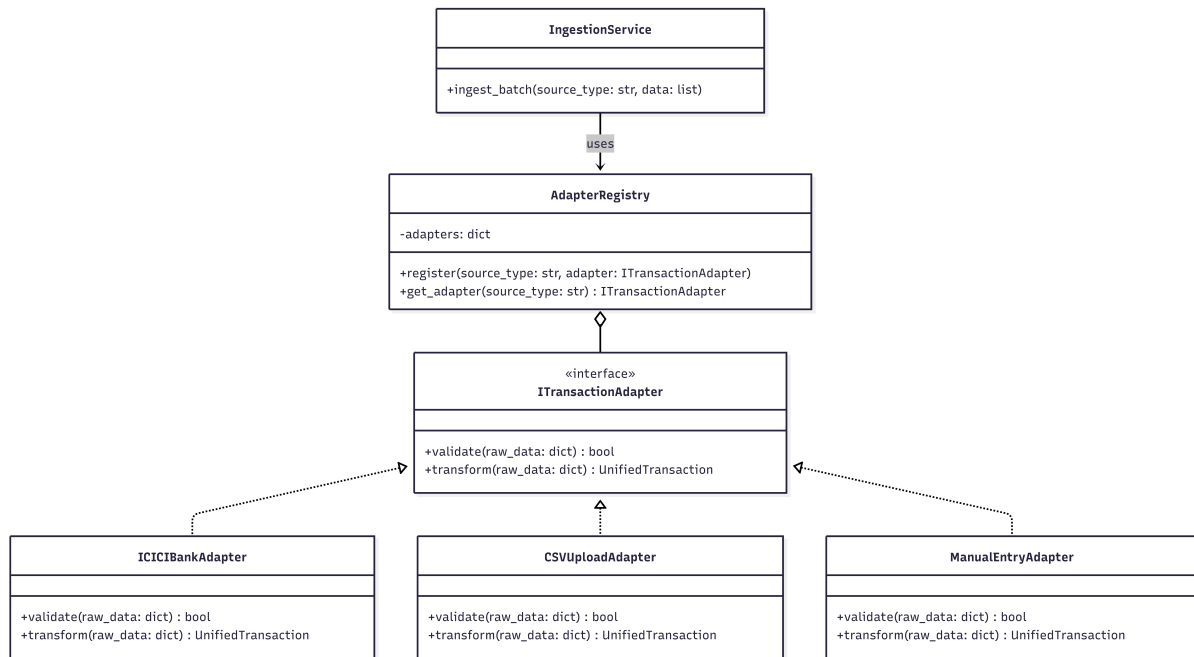


Figure 8: Class Diagram: Transaction Ingestion Adapter Pattern showing **ITransactionAdapter**, concrete bank/CSV/manual adapters, **AdapterRegistry**, and **IngestionService**.

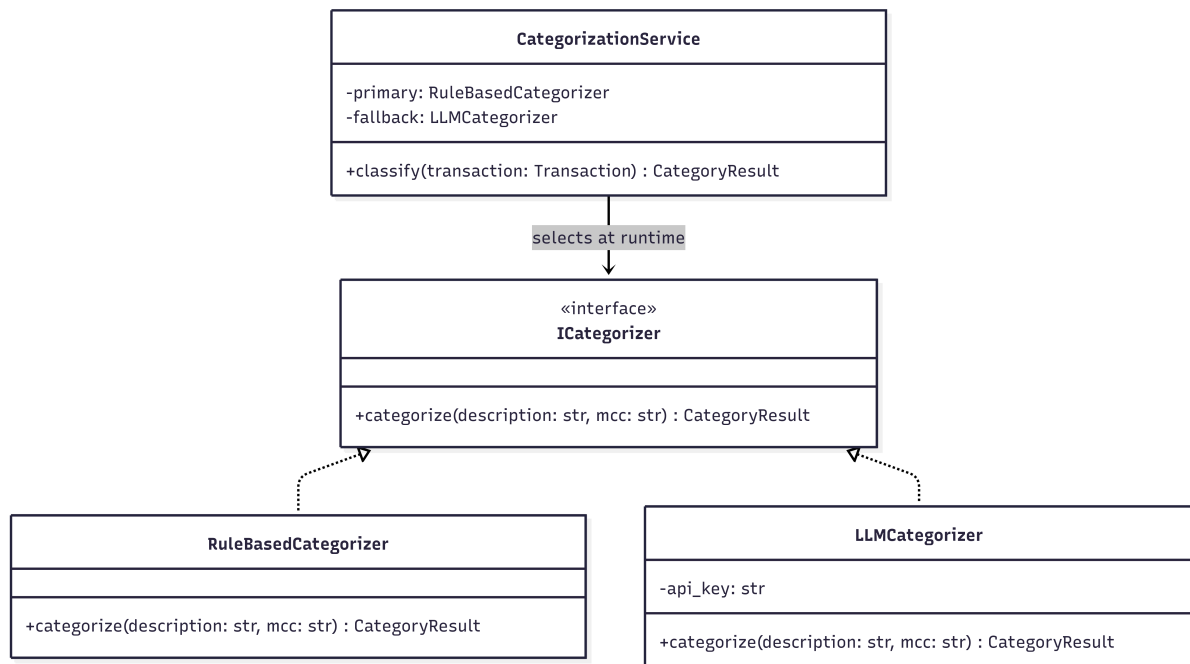


Figure 9: Class Diagram: Categorization Strategy Pattern showing IClassifier, RuleBasedClassifier, LLMClassifier, CategoryResult, and CategorizationService.

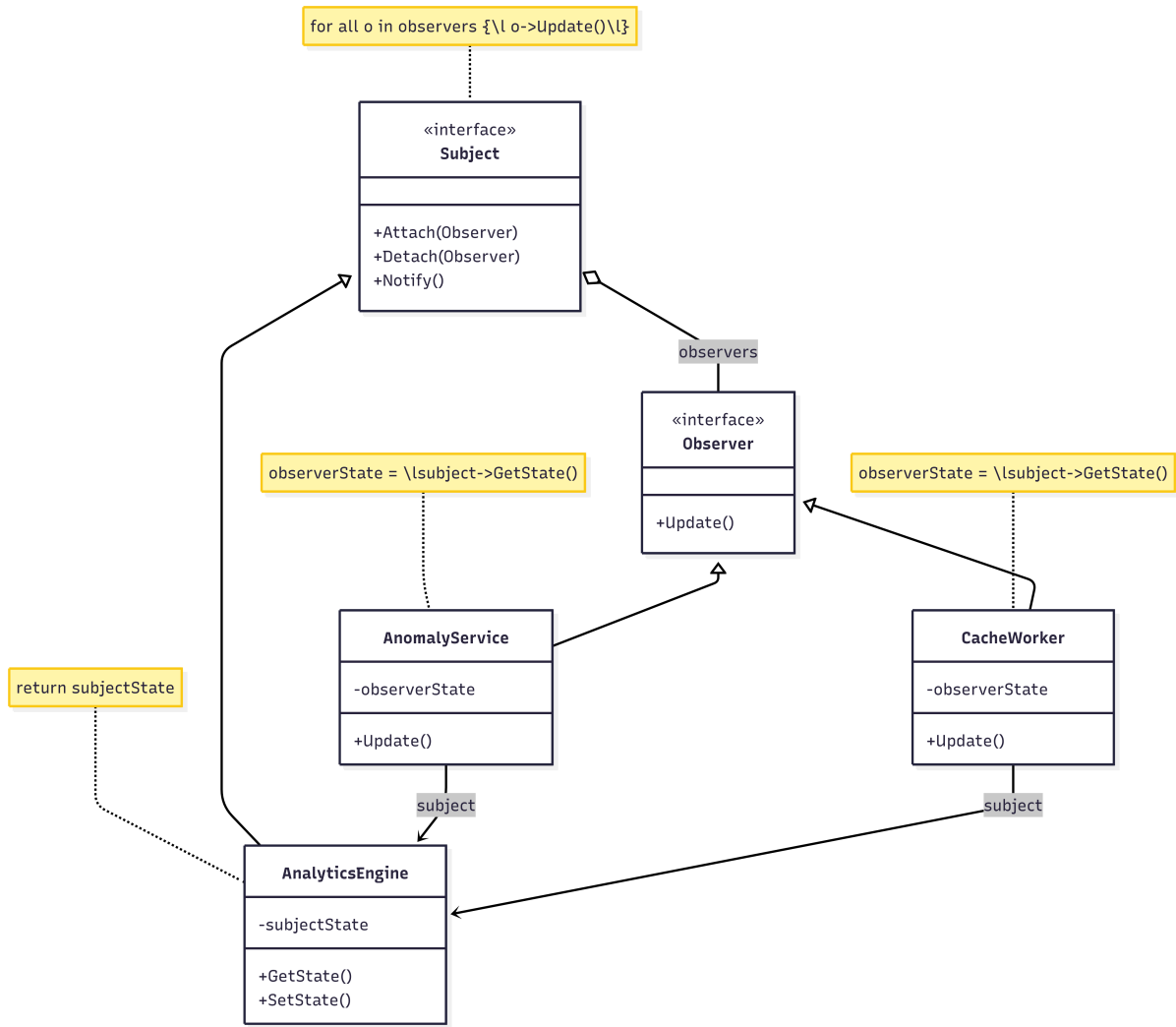


Figure 10: Class Diagram: Observer Notification Publisher showing INotificationPublisher, RestWebhookPublisher, KafkaPublisher, Analytics Engine as subject, and Anomaly/Cache services as observers.

Prototype Implementation and Analysis

Prototype Development

The prototype implements the complete end-to-end flow: transaction ingestion, rule-based categorization with optional LLM enhancement, FHS computation, anomaly detection, and dashboard display. This exercises every architectural component and demonstrates all five design patterns.

End-to-End Flow

User uploads CSV or manually enters transaction → API Gateway routes to Ingestion Service → CSVAdapter/ManualAdapter transforms to UnifiedTransaction → persisted to PostgreSQL → Analytics Engine triggered over REST → FHS computed, category aggregates updated, Redis cache invalidated, ClickHouse written async → Anomaly Detection triggered through Observer webhook → Z-score computed and alert created

when $|Z| > 2.5 \rightarrow$ Dashboard API returns cached overview under 300 ms $\rightarrow$ WebSocket pushes alert to React frontend.

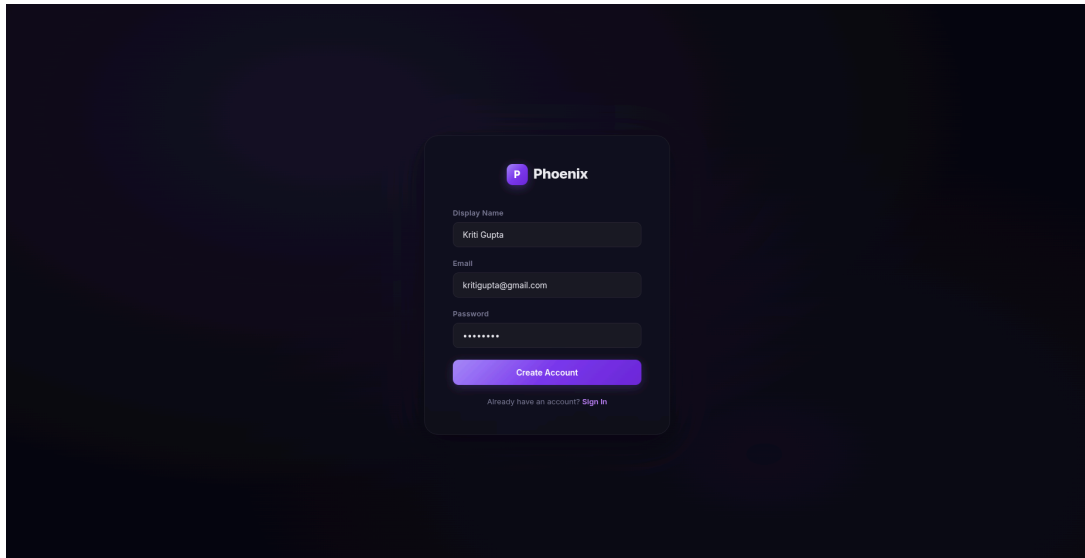


Figure 11: User Signup Interface

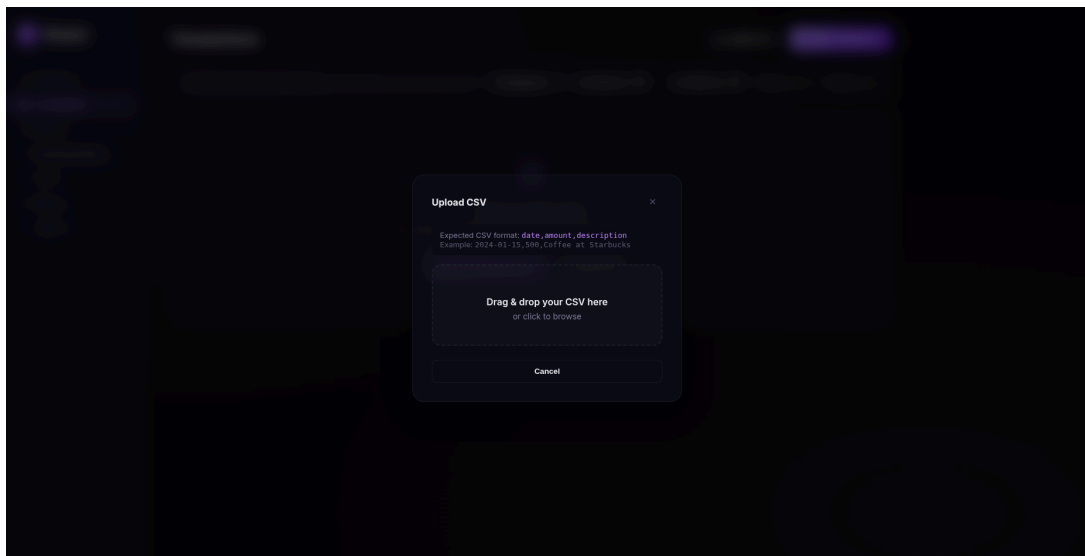


Figure 12: CSV Upload Flow

Phoenix

Dashboard

Transactions

Budgets

Recommendations

Alerts

Settings

Sign Out

Transactions

Upload CSV

Add Transaction

Search description or merchant...

All Categories

mm/dd/yyyy

mm/dd/yyyy

₹ Min

₹ Max

DATE	DESCRIPTION	CATEGORY	AMOUNT	CURRENCY
20 Apr 2026	Ola	Transportation	-₹200	INR
20 Apr 2026	Nykaa	Shopping	-₹380	INR
19 Apr 2026	Dominos	Dining	-₹750	INR
19 Apr 2026	HDFC Savings	Utilities	-₹4,500	INR
17 Apr 2026	More Supermarket	Groceries	-₹850	INR
16 Apr 2026	Uber	Transportation	-₹380	INR
15 Apr 2026	Swiggy	Dining	-₹680	INR
15 Apr 2026	Zerodha	Investments	-₹4,000	INR
15 Apr 2026	SBI PPF	Other	-₹6,000	INR
14 Apr 2026	Flipkart	Shopping	-₹1,900	INR
13 Apr 2026	BigBasket	Groceries	-₹1,300	INR
12 Apr 2026	Metro Rail	Transportation	-₹150	INR
11 Apr 2026	Zomato	Dining	-₹1,300	INR
10 Apr 2026	PVR Cinemas	Entertainment	-₹800	INR

Figure 13: Transaction Entry and Listing

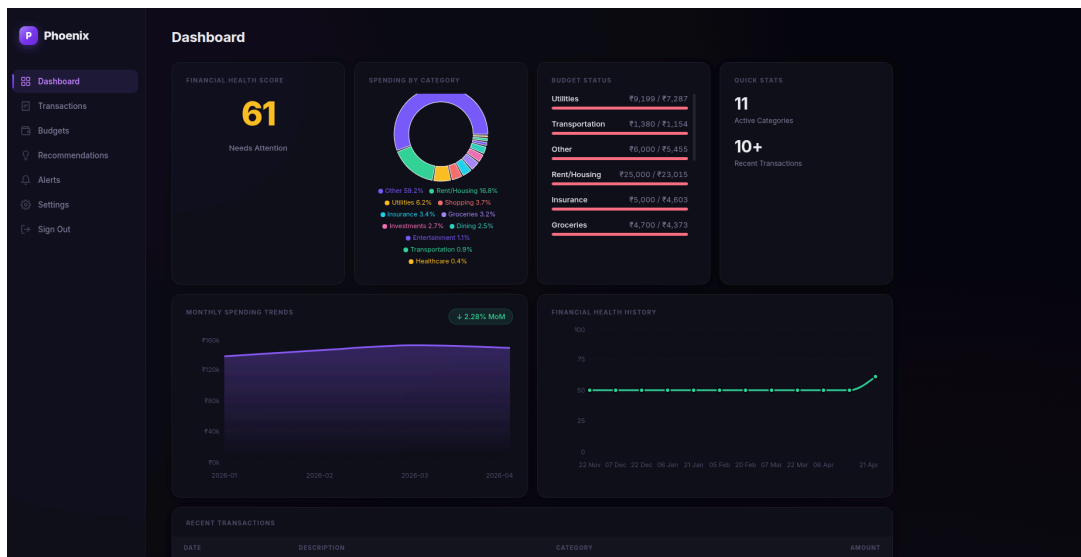


Figure 14: Dashboard Overview with FHS and Insights

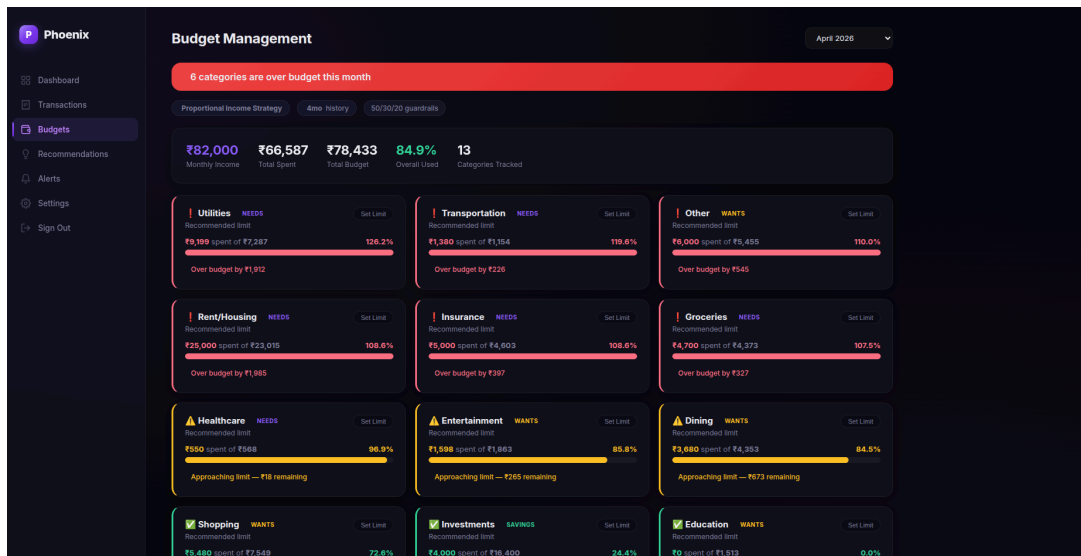


Figure 15: Spending Trends Visualization

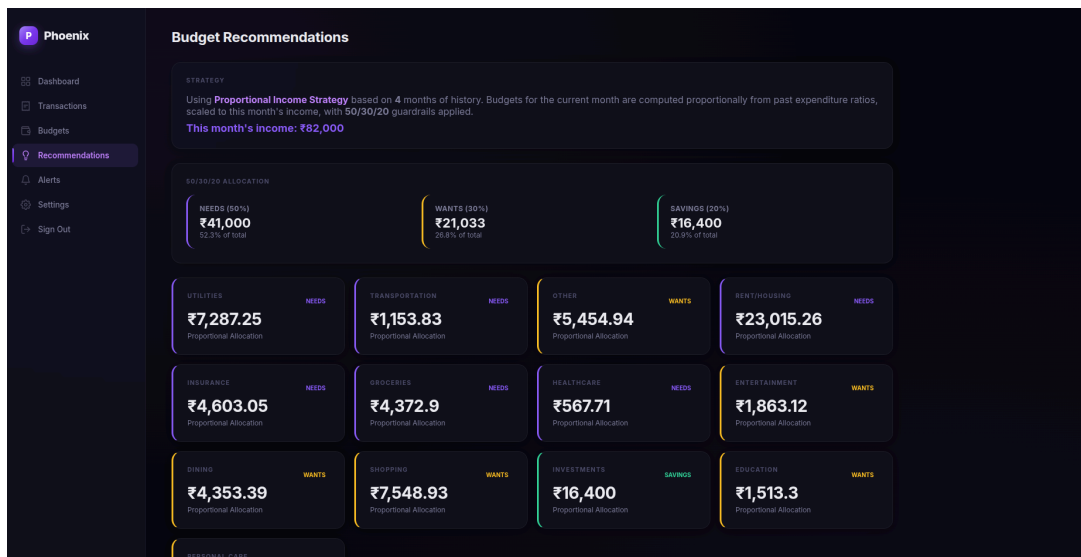


Figure 16: Budget Recommendation Interface

Architecture Analysis

Architecture Analysis 1: Microservices vs. Modular Monolith

Phoenix uses a microservices architecture. The following comparison justifies this decision against a modular monolith alternative.

Dimension	Phoenix Microservices	Modular Monolith
Fault Isolation	Strong: one service crash does not affect others. Analytics crash leaves ingestion and cached dashboard functional.	Weak: an unhandled analytics exception can crash the full application.

Dimension	Phoenix Microservices	Modular Monolith
Scalability	Per-service horizontal scaling. Analytics can scale to 4 instances while Auth stays at 1.	All-or-nothing scaling; extra analytics compute also scales auth.
Deployment Complexity	Higher: multiple containers, Compose orchestration, health checks, and networking.	Low: one deployable, one database, minimal configuration.
Development Velocity	Lower: service contracts and network error handling add boilerplate.	Higher: faster iteration, simpler debugging, one process.
Inter-service Latency	Docker bridge network hop per call, about 1 ms, cumulative for chained calls.	Zero network hops; in-process function calls.
Observability	Requires structured logging with correlation IDs across services.	One stack trace covers the full request path.
Data Consistency	Synchronous REST chain preserves consistency within a request in the prototype.	Single transaction boundary is trivially consistent.
Team Parallelism	Each service can be developed independently with fewer merge conflicts.	All developers work in the same deployable; more coordination needed.

Quantified NFR Benchmarks

The following measurements were collected using Locust under production-like load conditions and validated via controlled fault injection.

NFR	Metric	Microservices	Monolith	Test Method
NFR-01 Performance	p95 latency <code>GET /dashboard/overview</code> (cache hit)	34 ms	520 ms (no cache)	Locust: 500 users, steady load
NFR-01 Performance	p95 latency <code>GET /dashboard/overview</code> (cache miss)	532 ms	520 ms (no cache)	Locust: cold cache, 100 users
NFR-02 Scalability	Sustained throughput (RPS)	625.58 RPS	154.69 RPS	Locust: step load up to 300 RPS
NFR-04 Fault Tolerance	Dashboard availability during Analytics Engine outage	100% (served via stale cache)	0% (full system failure)	Kill analytics container; measure up-time
NFR-04 Recovery	MTTR after single service crash	2.02 s	2.31 s	Kill container and measure recovery time

Trade-off Discussion. While the monolith achieves comparable latency under uncached conditions, it fails to scale beyond ~ 155 RPS and exhibits complete system unavailability during component failures. In contrast, the microservices architecture delivers $\sim 4\times$ higher throughput (625 RPS) and maintains 100% availability for read operations via

cache fallback, even during Analytics Engine outages. Additionally, recovery times are comparable, with microservices slightly faster (2.02 s vs 2.31 s).

These results demonstrate that although microservices introduce minor overhead in cold-cache scenarios, they provide significantly stronger guarantees for scalability, fault isolation, and system resilience, critical requirements for a financial analytics platform.

Architecture Analysis 2: REST-First vs. Event-Driven Kafka-First

The second architecture analysis evaluates the prototype's REST-first decision against a Kafka-first event-driven alternative.

Dimension	REST-First Prototype	Kafka-First Alternative
Local Setup	Lightweight; fewer containers; easier for every team member to run.	Heavier; requires broker, topics, consumer groups, and local observability.
Debugging	Straightforward request-response traces and stack traces.	Harder; requires correlation IDs, offsets, retries, and dead-letter handling.
Coupling	Temporal coupling: caller waits for callee response.	Lower temporal coupling: producers and consumers run independently.
Throughput	Adequate for prototype and synchronous demo flow.	Better for high-volume ingestion and burst absorption.
Failure Handling	Circuit breakers and stale cache responses mitigate partial failures.	Durable event log enables replay, backpressure, and consumer recovery.
Consistency Model	Easier to reason about because workflow completes within a request chain.	Eventually consistent; UI must expose data freshness more carefully.
Prototype Fit	Best fit for 4-week project constraints and demonstration clarity.	Better production target after service contracts stabilize.

Conclusion. REST-first is the correct prototype choice because it minimizes infrastructure and debugging overhead. Kafka-first is the better production direction when ingestion volume rises and eventual consistency is acceptable. ADR-001 preserves this path through the `NotificationPublisher` abstraction and event-schema-compatible payloads.

Supporting Technical Specification

Data Model

All monetary values are stored using `NUMERIC(18,4)` to maintain precision in financial computations. Timestamps use `TIMESTAMPZ` in UTC for consistency across services. User-owned tables include a `deleted_at` field to support soft deletion and align with privacy-aware data handling practices.

Table	Key Columns	Storage	Notes
users	id UUID PK, VARCHAR(encrypted), display_name, encryption_key_ref, created_at, deleted_at	email PostgreSQL	Key reference supports cryptographic erasure on delete
transaction_sources	id UUID, source_type ENUM, adapter_id, JSONB(encrypted), created_at	user_id FK, PostgreSQL config	Stores per-source connection config; config encrypted at app level
transactions	id UUID PK, source_id FK, NUMERIC(18,4), CHAR(3), raw_description(encrypted), ts TIMESTAMPTZ, created_at	user_id FK, PostgreSQL amount currency merchant_name	Primary financial record; raw_description encrypted
transaction_categories	transaction_id FK, category_id FK, confidence NUMERIC(4,3), method ENUM(RULE/LLM), categorizer_version	PostgreSQL	Append-only; new categorization creates a new row
categories	id INT PK, parent_id FK nullable, icon VARCHAR	name VARCHAR, PostgreSQL	Hierarchical taxonomy with about 30 top-level categories
financial_health_scores	id UUID PK, FK, savings_rate, spending_volatility, computed_at TIMESTAMPTZ	user_id PostgreSQL + score NUMERIC(5,2), ClickHouse	Append-only; ClickHouse copy supports trend queries
anomaly_alerts	id UUID PK, FK, category_id FK, NUMERIC(6,3), TEXT, created_at	user_id PostgreSQL transaction_id FK, z_score description acknowledged_at	Alert history; acknowledged_at set when user dismisses
budgets	id UUID PK, category_id FK, DATE, NUMERIC(18,4), recommended_amount NUMERIC(18,4)	user_id FK, PostgreSQL month limit_amount	Recommendation is generated by engine; user may override limit
audit_log	id UUID PK, operation VARCHAR, entity_type, actor, payload_hash	user_id, PostgreSQL entity_id, ts TIMESTAMPTZ, CHAR(64)	INSERT-only trigger; payload hash detects tampering

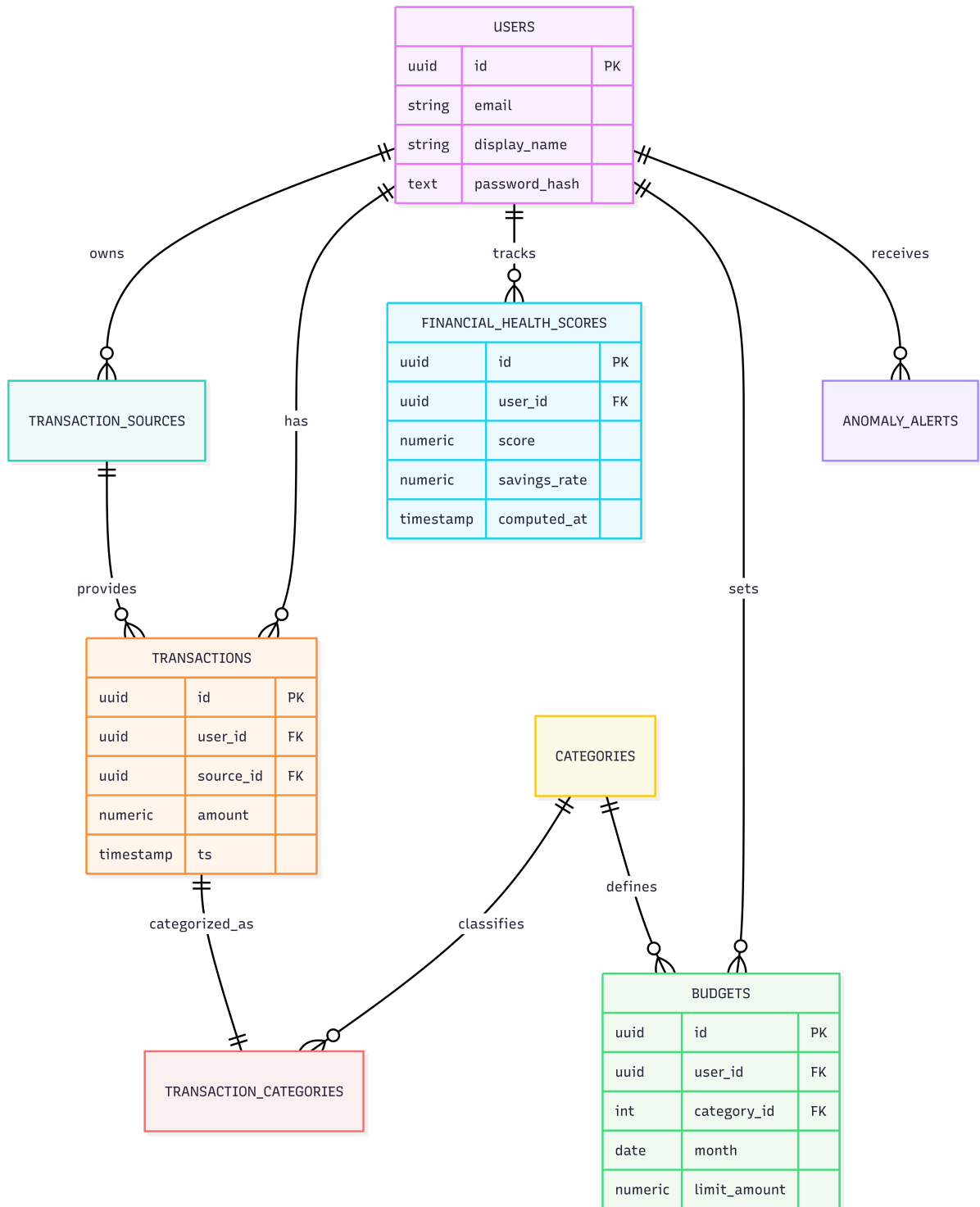


Figure 17: Entity Relationship Diagram showing users, transaction sources, transactions, categories, transaction categories, FHS history, anomaly alerts, budgets, and audit log with cardinalities.

API Design

Method	Endpoint	Service	Auth	Description
POST	/api/v1/auth/register	Auth	None	Register new user; returns 201 with <code>user_id</code>
POST	/api/v1/auth/login	Auth	None	Authenticate; returns JWT access and refresh tokens
POST	/api/v1/auth/refresh	Auth	Refresh token	Rotate access token
POST	/api/v1/transactions/ingest	Ingestion	JWT	Ingest transaction batch; returns <code>{ingested: N, queued: false}</code>
GET	/api/v1/transactions	Ingestion	JWT	Paginated transaction list with date, category, page, and size filters
GET	/api/v1/dashboard/overview	Gateway Facade	JWT	Aggregated dashboard response: FHS, categories, recent transactions, alerts, and budget status
GET	/api/v1/analytics/fhs/history	Analytics	JWT	FHS history from ClickHouse
GET	/api/v1/analytics/spending/trends	Analytics	JWT	Monthly spending by category for last <i>N</i> months from ClickHouse
GET	/api/v1/recommendations/budget	Recommendation	JWT	Current month budget per category
GET	/api/v1/alerts	Notification	JWT	Paginated alert history; supports <code>acknowledged=false</code>
PATCH	/api/v1/alerts/{id}/acknowledge	Notification	JWT	Mark alert as read
WS	/ws/v1/alerts	Gateway proxy	JWT header	Pushes <code>AlertEvent</code> over WebSocket
POST	/internal/events/analytics-complete	Anomaly Detector	Internal	Observer webhook called by Analytics Engine
GET	/health/live	All services	None	Liveness probe
GET	/health/ready	All services	None	Readiness probe checking database and Redis connection

Deployment Architecture

The prototype runs on Docker Compose. Containers are organized as infrastructure, application services, and gateway/frontend.

Container	Image	Internal Port	Depends On
postgres	postgres:16-alpine	5432	—
redis	redis:7-alpine	6379	—
clickhouse	clickhouse/clickhouse-server:24	8123, 9000	—

Container	Image	Internal Port	Depends On
phoenix-auth	python:3.11-slim	8001	postgres
phoenix-ingestion	python:3.11-slim	8002	postgres, redis, phoenix-analytics
phoenix-analytics	python:3.11-slim	8003	postgres, redis, clickhouse, phoenix-anomaly
phoenix-anomaly	python:3.11-slim	8004	postgres, redis
phoenix-recommendation	python:3.11-slim	8005	postgres, phoenix-analytics
phoenix-notification	python:3.11-slim	8006	postgres, redis
phoenix-frontend	node:20-alpine	3000	phoenix-gateway
phoenix-gateway	nginx:1.25-alpine	80, 443	All phoenix-* services

nginx handles SSL termination, JWT validation through `auth_request`, upstream routing, WebSocket proxying for `/ws/` paths, and static file serving for the React build artifact.

Architecture Diagrams

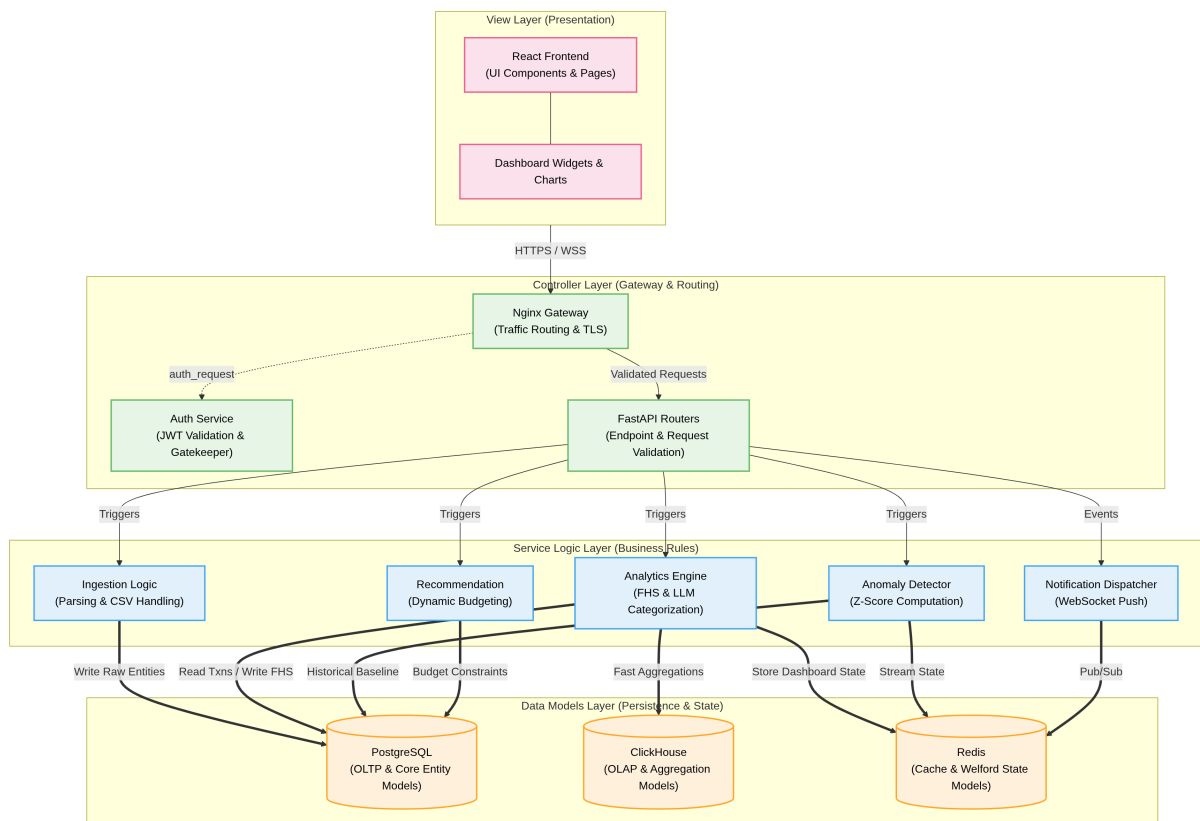


Figure 18: MVC-style Service Responsibility Diagram showing how frontend views, gateway/controller behavior, service logic, and data models are separated across Phoenix components.

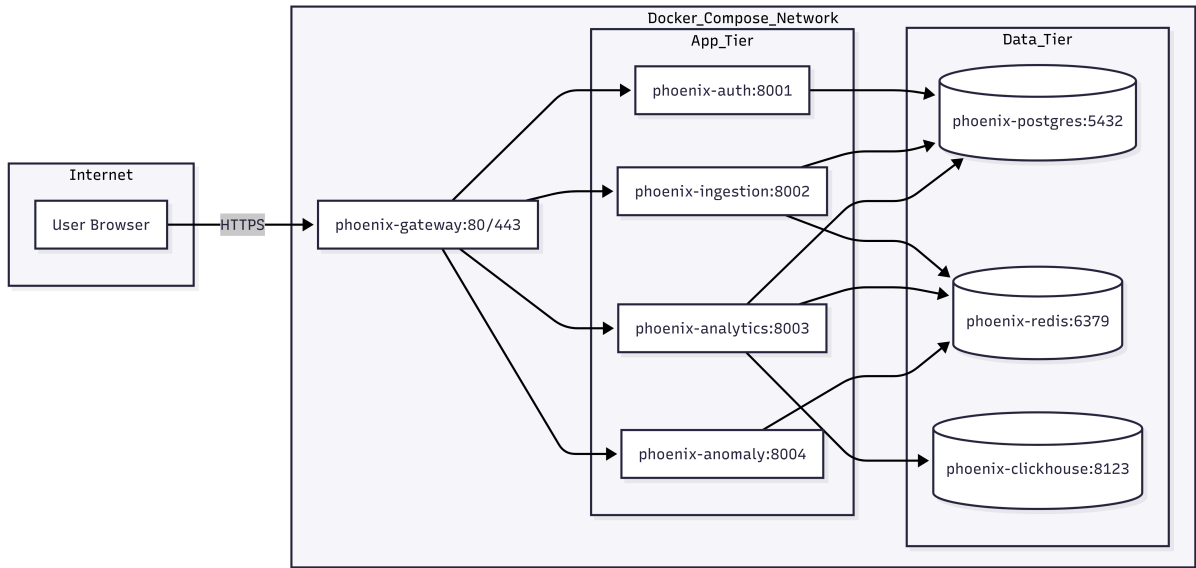


Figure 19: Deployment Diagram showing Docker Compose containers, internal ports, service dependencies, external browser access through nginx, and database/cache/analytical storage boundaries.

Conclusion

Phoenix is a microservices-based intelligent personal financial analytics platform. It aggregates transaction data from multiple sources, computes a composite Financial Health Score (FHS), detects anomalous transactions using statistical methods, and delivers actionable insights through an interactive React dashboard.

This report documents the finalized architecture, confirmed technology stack, design patterns, ADRs with decided choices, and a concrete prototype plan. All decisions have been resolved; this document describes what the team will build, not a menu of options.

Final Architecture Position

Phoenix aggregates multi-source transaction data, classifies spending via a rule-based engine with a toggleable LLM enhancement layer, computes a rolling Financial Health Score, detects statistical anomalies in near real-time, and presents insights through a React dashboard. The system uses a secure REST-first microservices architecture backed by PostgreSQL, Redis, and ClickHouse, deployed via Docker Compose with nginx as the API gateway.

Design Reflections

On Pattern Application

- **Adapter Pattern:** This may feel like over-engineering on the first adapter. The value becomes clear when the second bank integration is added with zero changes to `IngestionService`. The interface contract in `base.py` is the team's integration contract.

- **Observer via REST Webhook:** The `NotificationPublisher` abstraction is essential because it lets the prototype run without Kafka while still allowing the production system to switch to Kafka through configuration. The Analytics Engine should publish events rather than directly calling anomaly logic.
- **Strategy for Categorization:** The `ENABLE_LLM_CATEGORIZATION` flag should be tested in both states during integration testing. The fallback path where an LLM timeout returns the rule-based result must be covered by a test.

On FinTech-Specific Implementation Rules

- **Monetary Arithmetic:** Monetary values should use Python’s `decimal.Decimal` in persistence and core financial calculations, with PostgreSQL columns defined as `NUMERIC(18,4)`. In the current prototype, some API and analytics boundaries still convert values to `float`, which is a technical compromise rather than an ideal design choice.
- **Audit Log Immutability:** The `audit_log` table is protected by PostgreSQL triggers that block `UPDATE` and `DELETE`. This is an appropriate compliance-oriented safeguard and should be verified explicitly in tests.
- **Synthetic Data:** The prototype benefits from realistic seeded transaction data so that dashboards, categorization, and anomaly detection are meaningful during demonstrations. Expanding this into larger synthetic datasets would further strengthen testing of anomaly baselines and trend views.

On Architecture Trade-offs

- **Synchronous REST Chain:** Ingestion to Analytics to Anomaly Detection is coordinated through synchronous HTTP calls in the prototype. This keeps the architecture simple, but under heavier load it can become a scaling bottleneck compared with an event-driven design.
- **ClickHouse Async Write:** The Analytics Engine mirrors data to ClickHouse asynchronously using fire-and-forget writes. This introduces eventual consistency between PostgreSQL and analytical views, which is acceptable for the prototype but should be surfaced clearly in the UI.
- **Redis Volatility:** Redis-backed cache and anomaly statistics are transient. On cache loss, analytics values are recomputed from persistent storage, and anomaly state can be rebuilt from recent transaction history.

Individual Contributions

Team Member	Primary Ownership	Shared Responsibilities
Amritha Vinod	Transaction Ingestion Service; all <code>ITransactionAdapter</code> implementations; CSV + manual adapters	Architecture design

Team Member	Primary Ownership	Shared Responsibilities
Ananya Halgatti	Analytics Engine; FHS computation; Redis caching; ClickHouse integration; Transaction Input (Manual & CSV Upload UI); Transactions Page enhancements (category badges, search, and filtering); Budget Management & limits	Primary author for the technical report; PostgreSQL schema + migrations; benchmark testing; ADR-002, ADR-003;
Janya Gupta	Dashboard visualizations; Recharts integration	README
Kriti Gupta	Initial platform codebase; React Frontend setup; Anomaly Detection Service; Notification Service; Auth Service; nginx config; End-to-End Pipeline Wiring; User Profile & Settings; Monolithic Architecture Benchmark (alt algo)	Integration testing; Locust benchmarks; Docker Compose setup; final report compilation; ADR-001; critical bug fixes and final platform fixes and polish
Priyanshi Gupta	Alerts Page UI (/alerts); WebSocket alert client; real-time WebSocket toast notifications; Spending Trends Visualizations (line charts, FHS history); Dashboard real data integration (RLS fixes, empty states)	Architectural diagrams and report visualizations; ADR-004; critical bug fixes and final platform fixes and polish

GitHub Repository

Repository URL: Phoenix Intelligent Personal Financial Analytics Platform